

AACAP26 - OpenMP Challenge

Obiettivi e Regole

Questa challenge vi farà ragionare su vari argomenti che abbiamo trattato nel corso finora; in particolare, se torniamo ai risultati di apprendimento attesi presentati nella lezione introduttiva, allena le vostre capacità di

“sfruttare la conoscenza dell’architettura del calcolatore per migliorare le prestazioni dei programmi”.

Compilazione e consegna:

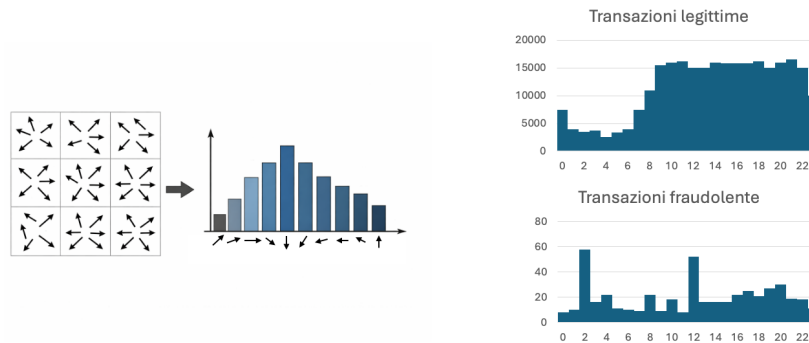
- Rispondete alle domande aggiungendo pagine in fondo a questo PDF (non cancellate la descrizione del problema).
- Salvate **due copie** del documento con le risposte, una con i nomi dei partecipanti al gruppo in una tabellina simile a quella in fondo a questa pagina e una *blind*.
- Mandate a serena.curzel@polimi.it due file:
 - Il documento PDF con i nomi;
 - Un archivio .zip contenente il documento PDF *blind* e tutto il codice ed eventuali script necessari per riprodurre gli esperimenti, sia nella versione base sia nell’eventuale versione ottimizzata.

Buon lavoro!

Nome e Cognome	Codice Persona

Descrizione del problema

Un istogramma può essere usato per identificare pattern in un dataset. Nella versione più semplice, un istogramma viene costruito analizzando il valore di ciascun elemento del dataset e incrementando il contatore corrispondente. Per esempio, la parte sinistra della figura qui sotto è un istogramma che identifica l'orientamento dei contorni rilevati in un'immagine; nella parte destra due istogrammi mostrano pattern di attività diversi nel corso di una giornata per transazioni bancarie legittime e fraudolente:



Possiamo dividere i dati di input in sezioni indipendenti, che quindi possono essere gestite in parallelo. Ecco uno snippet di codice che utilizza OpenMP per generare un istogramma che conta il numero di occorrenze di ogni lettera in una stringa di caratteri, raggruppandole in sette *bin* da quattro lettere (a-d, e-h, i-l, m-p, q-t, u-x, y-z):

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();

    #pragma omp for
    for (int i = 0; i < length; ++i) {
        int alphabet_pos = (int)data[i] - 'a';
        if (alphabet_pos >= 0 && alphabet_pos < 26) {
            int bin = alphabet_pos / 4;
            local_histograms[bin][tid]++;
        }
    }
}

for (int b = 0; b < 7; ++b)
    for (int t = 0; t < total_threads; ++t)
        histogram[b] += local_histograms[b][t];
```

Domande

1. **Cosa manca per poter eseguire il programma?** Completate il codice con l'inizializzazione corretta delle variabili, e aggiungete la misurazione del tempo totale di esecuzione.
2. **Il tempo di esecuzione diminuisce all'aumentare del numero di thread utilizzati?** Eseguite dei test con diversi numeri di thread, elencate le caratteristiche della macchina che avete usato per eseguirli (sistema operativo, numero di core, versione di OpenMP), e riportate in una tabella o in un grafico i risultati.
3. **Come mai la diminuzione del tempo di esecuzione non è lineare?** Fate delle ipotesi in base alle vostre conoscenze dell'architettura del calcolatore.
4. **Si può rendere il codice più efficiente?** In base alla risposta che avete trovato per il punto 3, modificate il programma e ripetete gli esperimenti del punto 2 per dimostrare il miglioramento nel tempo di esecuzione.